**CENTRO UNIVERSITÁRIO SAGRADO CORAÇÃO - UNISAGRADO**

Bacharelado em Ciência da Computação

Teoria dos Grafos - Prof. Patrick Pedreira

# TransitGraph

## Análise de rotas e conectividade da rede de ônibus de São Paulo com BFS e DFS

*Relatório técnico do projeto prático em grupo*

**Integrantes**

[PREENCHER: Nome completo - RA]

Matheus Leonardi - [PREENCHER: RA]

Richard Peghin da Silva - [PREENCHER: RA]

[PREENCHER: Nome completo - RA]

[PREENCHER: Nome completo - RA]

Repositório: github.com/MatheusLeonardi/TransitGraph

Bauru - novembro de 2026

# Sumário

1 Identificação do projeto
2 Descrição e relevância do problema
3 Origem e características dos dados
4 Modelagem como grafo
5 Fundamentação e descrição dos algoritmos
6 Geometria dos itinerários e coerência viária
7 Arquitetura e implementação
8 Metodologia de testes
9 Resultados obtidos
10 Limitações e possibilidades de melhoria
11 Decisões de escopo
12 Contribuições individuais
13 Referências
14 Declaração de uso de ferramentas de Inteligência Artificial

# 1 Identificação do projeto

| Item | Descrição |
|---|---|
| Projeto | TransitGraph - análise de rotas e conectividade da rede de ônibus de São Paulo |
| Tema | Tema 1 - representação e busca em grafos |
| Algoritmos obrigatórios | Busca em Largura e Busca em Profundidade, implementadas pelo grupo |
| Dados | Feed GTFS oficial da SPTrans, versão de 31/10/2026 |
| Tecnologias | Python, Flask, SQLite, HTML, CSS, JavaScript, Leaflet e Cytoscape.js |
| Repositório | github.com/MatheusLeonardi/TransitGraph |

# 2 Descrição e relevância do problema

O feed utilizado descreve 22.271 pontos de parada e 1.348 linhas de ônibus. Em uma rede desse tamanho, perguntas sobre caminhos, alcançabilidade e conectividade não podem ser respondidas por inspeção manual. O TransitGraph transforma os dados operacionais em um grafo e aplica BFS e DFS a uma instância real da rede de São Paulo.

A aplicação permite encontrar um caminho com o menor número de conexões entre duas paradas, comparar a ordem de visita das buscas, verificar quais paradas são alcançáveis e examinar componentes fracamente conexas. O projeto usa os algoritmos como parte central da solução, e não como demonstrações isoladas.

Objetivos específicos:

- usar BFS para obter o caminho com o menor número de arestas em um grafo não ponderado;
- usar DFS para explorar a rede em profundidade e analisar alcançabilidade;
- mostrar a ordem de processamento dos vértices e comparar fila e pilha;
- representar os resultados em mapa e subgrafo interativo;
- validar a solução com testes automatizados e medições reproduzíveis.

# 3 Origem e características dos dados

## 3.1 Fonte

Os dados vêm do feed GTFS Schedule publicado pela SPTrans. O arquivo ZIP contém tabelas relacionadas com paradas, linhas, viagens, calendários, sequências de paradas e geometrias dos itinerários. O arquivo usado foi obtido em 31/10/2026 e permanece no repositório para permitir a reprodução da instância.

## 3.2 Arquivos e volume

| Arquivo | Registros | Uso no projeto |
|---|---|---|
| stops.txt | 22.271 | Paradas com nome e coordenadas |
| routes.txt | 1.364 | 1.348 linhas de ônibus e 16 linhas de trilhos |

| Arquivo | Registros | Uso no projeto |
|---|---|---|
| trips.txt | 2.270 | Viagens associadas a linha, sentido, calendário e shape |
| stop_times.txt | 99.064 | Sequência de paradas usada para gerar as arestas |
| calendar.txt | 6 | Dias de operação dos serviços |
| frequencies.txt | 40.374 | Intervalos entre partidas, não usados no grafo |
| shapes.txt | 1.149.364 | Traçado geográfico dos itinerários |

### 3.3 Particularidades e limitações

- O importador seleciona apenas route_type 3, correspondente aos ônibus.
- Plataformas de um mesmo terminal e paradas dos dois lados da rua mantêm stop_id distintos.
- O grafo representa os serviços ativos na data escolhida e não incorpora alterações em tempo real.
- Frequências, trânsito, tarifa, tempo de viagem e caminhada não entram no custo da busca.

## 4 Modelagem como grafo

A rede é representada por G = (V, E). Cada vértice de V corresponde a uma parada de ônibus. Existe uma aresta dirigida u para v quando uma viagem ativa passa por u e, imediatamente depois, por v. A lista de adjacência preserva a direção e evita arestas duplicadas.

| Elemento | Definição adotada |
|---|---|
| Vértice | Parada de ônibus identificada por stop_id |
| Aresta | Conexão consecutiva entre duas paradas de uma viagem ativa |
| Direção | Definida pela ordem da viagem no GTFS |
| Peso | Não ponderado; cada aresta tem custo unitário |
| Atributos | Linhas, viagem representativa, shape e sentido |
| Representação | Lista de adjacência com uso de memória O(V + E) |

O grafo é direcionado porque a existência do trecho u para v não garante o trecho inverso. Essa escolha representa itinerários de ida e volta distintos e a ordem oficial das viagens. A análise de componentes fracas cria temporariamente uma visão não direcionada sem alterar o grafo principal.

### 4.1 Instância principal

| Medida | Valor |
|---|---|
| Data de serviço | 01/11/2026 |
| Paradas | 20.812 |
| Arestas direcionadas | 27.218 |

| Medida | Valor |
|---|---|
| Associações linha-aresta | 76.711 |
| Viagens ativas | 1.774 |
| Componentes fracamente conexas | 4 |
| Maior componente | 20.797 paradas |

# 5 Fundamentação e descrição dos algoritmos

## 5.1 Busca em Largura

A Busca em Largura explora o grafo por níveis. Uma fila FIFO armazena os vértices descobertos. Em um grafo não ponderado, a primeira remoção do destino da fila determina um caminho com o menor número de arestas. O vetor de predecessores permite reconstruir esse caminho.

```
BFS(G, origem, destino)
  fila <- [origem]; descobertos <- {origem}
  enquanto fila não estiver vazia:
    u <- remover_inicio(fila); registrar u na ordem de visita
    se u = destino: interromper
    para cada vizinho v de u ainda não descoberto:
      pai[v] <- u; adicionar v ao fim da fila
```

A implementação usa collections.deque. A complexidade é O(V + E) no tempo e O(V) na memória.

## 5.2 Busca em Profundidade

A Busca em Profundidade segue um ramo até não encontrar novos vizinhos e então retrocede. A versão do projeto é iterativa e usa uma pilha LIFO, evitando o limite de recursão do Python em percursos longos.

```
DFS(G, origem, destino opcional)
  pilha <- [origem]; descobertos <- {origem}
  enquanto pilha não estiver vazia:
    u <- remover_topo(pilha); registrar u na ordem de visita
    se u = destino: interromper
    empilhar em ordem inversa os vizinhos ainda não descobertos
```

A DFS visita todos os vértices alcançáveis quando não recebe destino. Sua complexidade também é O(V + E). Diferentemente da BFS, ela não garante o menor caminho.

## 5.3 Comparação

| Característica | BFS | DFS |
|---|---|---|
| Estrutura | Fila FIFO | Pilha LIFO |
| Exploração | Por níveis | Aprofunda um ramo |
| Menor caminho não ponderado | Garante | Não garante |
| Uso no projeto | Rota mínima em arestas | Alcançabilidade e comparação |
| Complexidade | O(V + E) | O(V + E) |

A API devolve a ordem de visita, o total de vértices processados, o total descoberto, o caminho e o tempo da execução. A interface apresenta as primeiras 200 visitas e conserva a contagem total para manter a página legível.

# 6 Geometria dos itinerários e coerência viária

BFS e DFS operam sobre o grafo de paradas. A geometria desenhada no mapa é uma etapa posterior e não altera os caminhos encontrados pelos algoritmos.

## 6.1 Associação entre paradas e shapes

Para cada viagem ativa, o importador associa a sequência de paradas aos pontos do shape correspondente. A associação usa programação dinâmica para preservar a ordem de percurso e evitar erros em trechos nos quais a linha cruza ou retorna por uma via próxima. Uma associação é rejeitada quando uma parada fica a mais de 150 metros do ponto escolhido.

| Medida de geometria | Valor |
|---|---|
| Shapes ativos | 1.774 |
| Pontos de shape persistidos | 903.857 |
| Associações validadas | 76.651 |
| Associações rejeitadas | 60 |

## 6.2 Refinamento viário

Quando o shape contém intervalos maiores que 60 metros, a aplicação pode consultar o OSRM para ajustar a polilinha à malha viária do OpenStreetMap. O resultado é aceito somente quando respeita limites de confiança, comprimento, continuidade e proximidade com o shape GTFS. Resultados incoerentes ou falhas de rede mantêm integralmente a geometria oficial.

As respostas aceitas e rejeitadas ficam em cache no SQLite. Esse refinamento melhora a visualização das ruas, mas não comprova sozinho todas as regras locais de circulação. A direção das arestas continua sendo definida pela sequência das viagens no GTFS.

# 7 Arquitetura e implementação

## 7.1 Visão geral

1. Pré-processamento: leitura do ZIP GTFS, filtragem por data, construção do grafo, associação das geometrias e gravação em SQLite.
2. Backend: aplicação Flask que carrega o grafo em memória e oferece uma API REST.
3. Frontend: página com Leaflet, OpenStreetMap, Cytoscape.js e painéis para buscas e conectividade.

## 7.2 Principais rotas da API

| Rota | Método | Função |
|---|---|---|
| /api/stats | GET | Estatísticas e metadados da instância |

| Rota | Método | Função |
|---|---|---|
| /api/stops | GET | Pesquisa de paradas por nome ou código |
| /api/stops/map | GET | Paradas dentro da área visível |
| /api/algorithms/bfs | POST | Caminho BFS e ordem de visita |
| /api/algorithms/dfs | POST | Caminho ou alcançabilidade por DFS |
| /api/algorithms/compare | POST | Comparação BFS e DFS |
| /api/connectivity | GET | Componentes fracamente conexas |
| /api/graph/neighborhood | GET | Subgrafo de uma parada |

### 7.3 Instalação e execução

```
git clone https://github.com/MatheusLeonardi/TransitGraph.git
cd TransitGraph
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python run.py
python -m pytest -v
```

## 8 Metodologia de testes

A suíte automatizada possui 30 testes. Eles cobrem casos pequenos verificáveis, entradas inválidas, ausência de caminho, ordem de visita, direção das arestas, componentes fracas, persistência no SQLite, integração da API, associação com shapes e validação do refinamento viário.

| Grupo | Exemplos verificados |
|---|---|
| BFS | distância mínima, origem igual ao destino, ausência de caminho e ordem de visita |
| DFS | alcançabilidade, caminho em profundidade e ordem de visita |
| Grafo | direção, arestas duplicadas, metadados e shapes representativos |
| Persistência | linhas, geometrias compactadas e metadados de construção |
| API | comparação entre algoritmos e validação das entradas |
| Geometria | ordem no shape, rejeição por distância, cache e fallback |

```
python -m pytest -q
30 passed
```

O benchmark usa semente 42, 30 pares aleatórios no grafo real e cinco repetições por grafo sintético. O script registra o sistema, o processador, a versão do Python e os parâmetros da medição.

# 9 Resultados obtidos

## 9.1 Interface

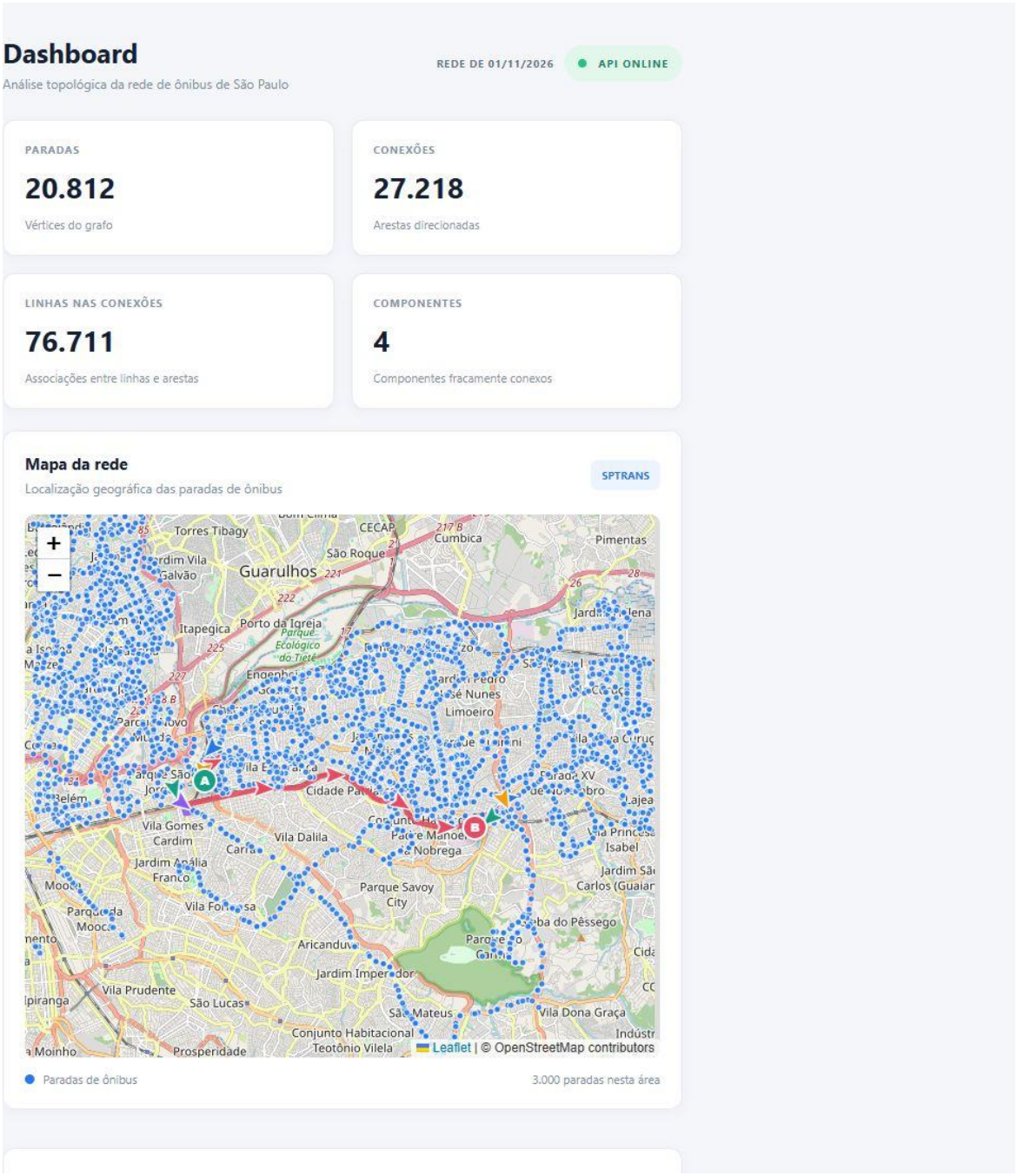



*Figura 1 - Interface atual com estatísticas, mapa e itinerário refinado.*

### 9.2 Comparação prática entre BFS e DFS

Na consulta entre as paradas 600012584 e 3609172, a BFS encontrou um caminho de 34 arestas depois de processar 3.258 vértices. A DFS encontrou o mesmo destino por um percurso de 1.127 arestas e processou 6.996 vértices. O caso mostra que a DFS pode localizar um caminho válido, mas somente a BFS garante o menor número de arestas.

| Métrica | BFS | DFS |
|---|---|---|
| Distância do caminho | 34 arestas | 1.127 arestas |
| Vértices processados | 3.258 | 6.996 |
| Comportamento | exploração por níveis | aprofundamento de ramos |
| Garantia de mínimo | sim | não |

### 9.3 Desempenho

| Instância | V | E | BFS mediana | DFS mediana |
|---|---|---|---|---|
| Grafo real | 20.812 | 27.218 | 6,54 ms | 12,58 ms |
| Sintético | 1.000 | 1.997 | 0,45 ms | 0,51 ms |
| Sintético | 5.000 | 9.997 | 2,63 ms | 2,93 ms |
| Sintético | 20.000 | 39.997 | 10,91 ms | 11,73 ms |
| Sintético | 50.000 | 99.997 | 26,72 ms | 30,20 ms |
| Sintético | 100.000 | 199.997 | 56,85 ms | 67,07 ms |
| Sintético | 200.000 | 399.997 | 134,78 ms | 164,84 ms |

As medições foram realizadas em 02/11/2026 com Python 3.14 em Windows 11 e processador AMD64 Family 25 Model 33. Os tempos dependem do computador. O arquivo scripts/benchmark.py permite repetir o experimento na máquina da apresentação.

### 9.4 Conectividade

A instância possui quatro componentes fracamente conexas, com tamanhos 20.797, 11, 2 e 2. A maior componente contém aproximadamente 99,9% das paradas. Os três fragmentos menores devem ser interpretados à luz do cadastro de plataformas e das viagens ativas na data selecionada.

## 10 Limitações e possibilidades de melhoria

- A BFS minimiza arestas, não tempo, distância, tarifa ou número de baldeações.
- A escolha posterior de linhas reduz baldeações sem alterar o caminho de paradas encontrado pela BFS.
- Plataformas próximas não são agrupadas em um único vértice de estação.
- Metrô e trem não entram no grafo principal.
- O OSRM público depende de internet; a aplicação conserva o shape GTFS como fallback.

- A interface limita a ordem exibida às primeiras 200 visitas, embora a API preserve as contagens totais.
- Trânsito, horários em tempo real e caminhada exigiriam outro modelo e algoritmos ponderados.

## 11 Decisões de escopo

O Tema 1 exige BFS e DFS, e ambos são partes essenciais da aplicação. Pontes, pontos de articulação e simulação de fechamento por Tarjan não fazem parte dos algoritmos obrigatórios fornecidos no enunciado do tema. Caso essas funções tenham sido prometidas em uma proposta anterior, o grupo deve comunicar ao professor que o escopo final priorizou a comparação completa entre BFS e DFS na rede real.

A aplicação usa uma instância principal selecionada no pré-processamento pela data do serviço. A construção de outros dias é feita por linha de comando. Um seletor de instâncias na interface pode ser acrescentado como evolução se o enunciado completo o exigir.

## 12 Contribuições individuais

| Integrante | Frente | Contribuições |
|---|---|---|
| Matheus Leonardi | [PREENCHER] | [PREENCHER] |
| Richard Peghin da Silva | [PREENCHER] | [PREENCHER] |
| [PREENCHER] | [PREENCHER] | [PREENCHER] |
| [PREENCHER] | [PREENCHER] | [PREENCHER] |
| [PREENCHER] | [PREENCHER] | [PREENCHER] |

Cada integrante deve registrar suas contribuições reais no histórico Git e estar preparado para explicar o código, os algoritmos, os testes e as decisões apresentadas.

## 13 Referências

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Algoritmos: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012. Cap. 22.

SPTRANS. Área do Desenvolvedor - GTFS. Disponível em: https://www.sptrans.com.br/desenvolvedores/. Acesso em: nov. 2026.

GTFS. GTFS Schedule Reference. Disponível em: https://gtfs.org/documentation/schedule/reference/. Acesso em: nov. 2026.

PROJECT OSRM. Open Source Routing Machine. Disponível em: https://project-osrm.org/. Acesso em: nov. 2026.

OPENSTREETMAP. Copyright and License. Disponível em: https://www.openstreetmap.org/copyright. Acesso em: nov. 2026.

FRANZ, M. et al. Cytoscape.js: a graph theory library for visualisation and analysis. Bioinformatics, v. 32, n. 2, p. 309-311, 2016.

LEAFLET. Leaflet 1.9. Disponível em: https://leafletjs.com. Acesso em: nov. 2026.

PALLETS. Flask Documentation. Disponível em: https://flask.palletsprojects.com. Acesso em: nov. 2026.

# 14 Declaração de uso de ferramentas de Inteligência Artificial

O grupo declara o uso das ferramentas abaixo como apoio à revisão técnica, documentação e desenvolvimento. Todos os resultados devem ser conferidos pelos integrantes por leitura do código, execução dos testes e verificação da interface.

| Ferramenta | Etapa | Uso | Revisão pelo grupo |
|---|---|---|---|
| Claude (Anthropic) | Revisão inicial | Auditoria do repositório e rascunho do relatório | [PREENCHER: responsáveis e verificações] |
| Codex (OpenAI) | Implementação e revisão | Ordem de visita, shapes, refinamento viário, testes, benchmark e reconstrução do relatório | Leitura do código, 30 testes automatizados e verificação visual da interface |

Os integrantes devem revisar o conteúdo final e confirmar que conseguem explicar as partes desenvolvidas ou modificadas com auxílio de IA.